Procesos

Keywords: procesos,

¿Qué es un proceso (*task*)?

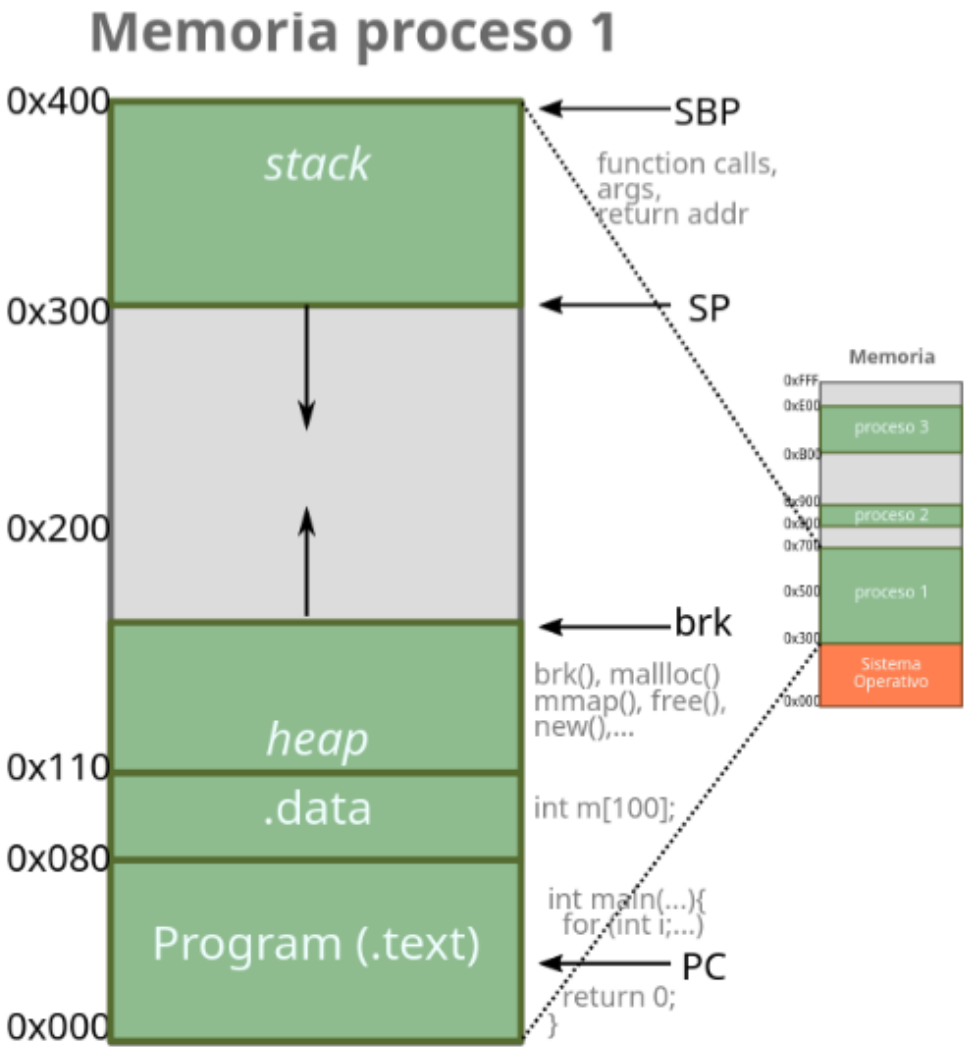
- Código en ejecución (código + recursos(memoria, archivos, sockets, I/O, etc))
- La multiprogramación permite mantener múltiples procesos en memoria.

La CPU puede atender los procesos en algún orden (*scheduling*), de manera que, si la CPU atiende "simultáneamente" varios procesos, entonces tenemos *multitasking*.

Multitasking involves overlapping and interleaving the execution of several programs. This is often achieved by capitalizing on the difference between a computer's rapid processing capacity and the slower rates of its input/output devices.

Composición y representación de un proceso

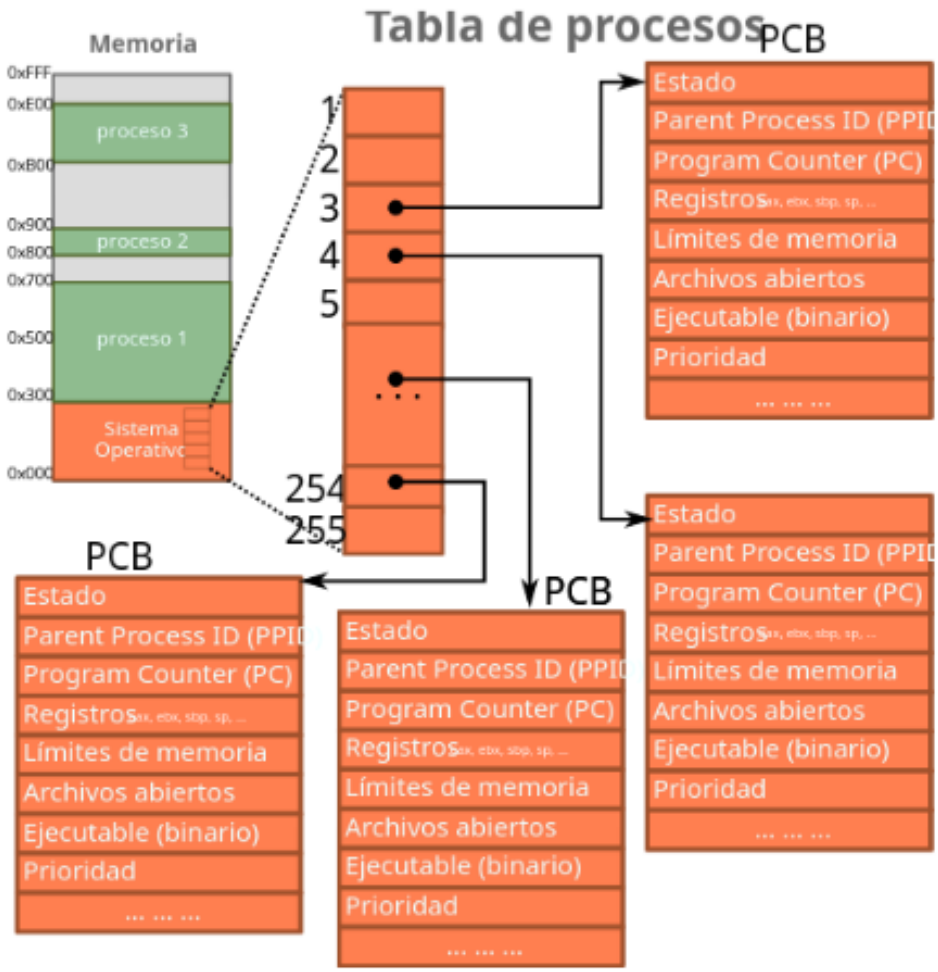
- Código (.text): informacipon estática
- Datos (.data): variables globales
- Heap: memoria asignada dinámicamente (en runtime)
- Stack: cada ítem del stack representa un llamado a función (call frame) y contiene:
    - Parámetros
    - Variables locales
    - Lugar de retorno (donde estaba anteriormente la ejecución: PC)
    - Stack Base Pointer SBP
    - Stack Pointer SP



El sistema operativo mantiene la Tabla de Procesos. Esta almacena la información de cada proceso en su respectivo Process Control Block (PCB), el que a su vez almacena:

- Estado del proceso
- PID
- PC
- Registros de CPU: estado de ejecución
- Información de scheduling: prioridades, tipo de cola, ...
- Información de memoria: límites, tabla de páginas/segmentos, ...
- Contabilidad (accounting)
- Información de I/O: archivos y dispositivos abiertos, ...

PROF

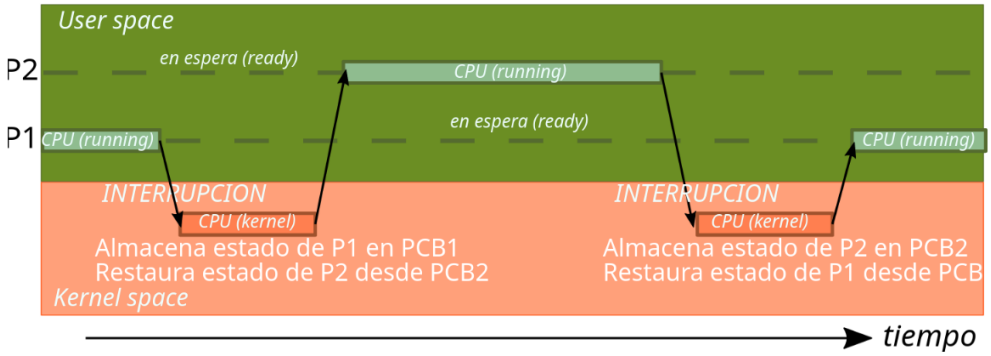


Multitasking

El cambio de procesos se conoce como context switch

- OS actúa luego de una interrupción (syscall, timer, evento, ...)
- OS almacena estado de registros de P1 en PCB1. P1 queda "en pausa"
- OS restaura estado de registros desde PCB2
- P2 continúa su ejecución

PROF



¿Cómo se crean los procesos?

Durante la inicialización del kernel se crea un proceso raíz con PID=1

- Linux: systemd, init, upstart

- MacOS: launchd
- Windows: InitialSystemProcess, System

Syscall fork()

¿Quién sigue ejecutando?

- Parent y Child siguen ejecutando concurrentemente
- Ambos existen en la memoria del computador
- Cuál proceso continúa en estado running depende de la implementación

¿Qué "personalidad" tiene el child?

- El hijo es un duplicado casi exacto del padre
- El hijo en un nuevo proceso (otro espacio de direcciones)
- El hijo copia la memoria del padre a su nuevo espacio
- El hijo y el padre continúan ejecutando desde el retorno del fork()

fork() retorna el PID del nuevo proceso al padre y retorna 0 al nuevo proceso.

Syscall exec()

- Carga un binario en memoria reemplazando el código de quien lo llamó, e inicia su ejecución.
- El programa nuevo se "roba" el proceso (la memoria es sobreescribible)
- Solamente retorna en caso de que falle.

Syscall wait()

Espera el término de un proceso hijo.

Syscall exit()

Termina el proceso con un código de retorno dado y lo entrega al padre.

PROF

Señales, huerfanos y zombies

Syscall kill()

- Envía una señal a otro proceso.
- kill –l permite ver las señales disponibles.
- SIGTERM indica al proceso que debe terminar.
- SIGKILL elimina al proceso de la tabla de procesos (sin piedad).

```
[francisco@archlinux ~]$ kill -l
 1) SIGHUP       2) SIGINT       3) SIGQUIT      4) SIGILL       5) SIGTRAP
 6) SIGABRT      7) SIGBUS       8) SIGFPE       9) SIGKILL     10) SIGUSR1
11) SIGSEGV     12) SIGUSR2     13) SIGPIPE     14) SIGALRM     15) SIGTERM
16) SIGSTKFLT   17) SIGCHLD     18) SIGCONT     19) SIGSTOP     20) SIGTSTP
21) SIGTTIN     22) SIGTTOU     23) SIGURG      24) SIGXCPU     25) SIGXFSZ
26) SIGVTALRM   27) SIGPROF     28) SIGWINCH    29) SIGIO       30) SIGPWR
31) SIGSYS      34) SIGRTMIN    35) SIGRTMIN+1  36) SIGRTMIN+2  37) SIGRTMIN+3
38) SIGRTMIN+4  39) SIGRTMIN+5  40) SIGRTMIN+6  41) SIGRTMIN+7  42) SIGRTMIN+8
43) SIGRTMIN+9  44) SIGRTMIN+10 45) SIGRTMIN+11 46) SIGRTMIN+12 47) SIGRTMIN+13
48) SIGRTMIN+14 49) SIGRTMIN+15 50) SIGRTMAX-14 51) SIGRTMAX-13 52) SIGRTMAX-12
53) SIGRTMAX-11 54) SIGRTMAX-10 55) SIGRTMAX-9  56) SIGRTMAX-8  57) SIGRTMAX-7
58) SIGRTMAX-6  59) SIGRTMAX-5  60) SIGRTMAX-4  61) SIGRTMAX-3  62) SIGRTMAX-2
63) SIGRTMAX-1  64) SIGRTMAX
```

Procesos huérfanos

Linux:

- Cuando un padre termina (exit) o muere (kill), sus hijos quedan huérfanos y pasan a ser hijos de init o systemd.
- Sin embargo, en condiciones complicadas puede parecer que los hijos si mueren).
- Init hace wait() periódicamente por sus hijos.

Procesos zombies

Linux:

- Cuando un proceso termina y su padre no hace wait()
- Proceso terminado no se borra inmediatamente de la tablade procesos
- El proceso tampoco ejecuta nada, porque terminó.
- Proceso queda en estado zombie hasta que el padre hace wait()